

# Final Project Report: Building Aethernet

## An Acoustic Network Protocol Stack Implementation

Jiayi Gu - ID: 2022533155 - Shanghaitech

Yi Gu - ID: 2022533136 - Shanghaitech

January 2026

### Abstract

This report summarizes the design and implementation of “Aethernet”, a complete network protocol stack using acoustic signals as the physical medium. Over four sequential projects, we evolved the system from simple wave generation to complex application-layer interactions, including DNS resolution and HTTP content fetching. We discuss our technical choices, such as FSK modulation and IP fragmentation, and share insights into the physical challenges of acoustic communication.

## 1 Disclosure Preference

Disclosure Preference: **Disclose content with names (100%)**

## 2 Project 1: The Physical Layer Foundation

In the initial stage, our primary goal was to establish a reliable acoustic link between two air-gapped nodes.

### 2.1 Modulation Schemes: BPSK vs. FSK

We experimented with two primary modulation methods: Binary Phase Shift Keying (BPSK) and Frequency Shift Keying (FSK). While BPSK theoretically offers better spectral efficiency, we found it extremely difficult to maintain phase synchronization over the air due to multipath effects and the inherent sampling clock frequency offset between sound cards.

Ultimately, we settled on **FSK** for its robustness. By mapping bits to distinct frequency peaks, our receiver could reliably distinguish signals even in the presence of moderate ambient noise.

### 2.2 A Physical Discovery: Frequency Absorption

A key highlight of Project 1 was our empirical study of frequency selection. Through trial and error, we discovered that **low-frequency signals (below 1kHz) were significantly absorbed by the wooden desks** in the lab environment. This absorption

caused massive attenuation, leading to a high Bit Error Rate (BER). Once we shifted our carrier frequencies to a higher range (4kHz - 10kHz), the stability improved drastically. We suggest that future tutorials include a note on the interaction between acoustic frequencies and physical surfaces.

## 3 Project 2 & 3: Addressing and Basic Connectivity

Projects 2 and 3 introduced the MAC layer and the Network layer.

### 3.1 Cable Connection and ICMP

Transitioning from air to audio cables simplified the noise environment but introduced new challenges in signal gain management. We successfully implemented the first tasks of these projects, establishing basic ICMP Echo (Ping) connectivity. By assigning private IP addresses (e.g., 10.0.0.1 and 10.0.0.2) to our Aethernet programs, we demonstrated that a layered protocol stack could operate over non-traditional media.

## 4 Project 4: Above IP - The Full Stack Journey

Project 4 was the most challenging yet rewarding phase, where we implemented DNS, TCP, and HTTP.

### 4.1 Recursive DNS Resolution

We configured NODE2 to act as a recursive DNS resolver. When NODE1 requested a domain like *www.baidu.com*, NODE2 utilized the host system's network stack to resolve the IP and relayed the response back via Aethernet. To ensure the TAs could verify the process via Wireshark, we added a trigger to NODE2 that initiated a real ICMP ping upon successful resolution, making the NAT and forwarding logic visible on the WLAN interface.

### 4.2 Transport Layer: TCP Handshake and Fragmentation

To support HTTP, we implemented a simplified TCP state machine.

- **Three-way Handshake:** We strictly followed the requirement of using an Initial Sequence Number (ISN) from a specific set. We chose 0x12345678.
- **IP Fragmentation (Bonus Task 5):** A critical hurdle was the 255-byte limit of our `FrameType` length field. The HTTP response for *example.com* was over 800 bytes. We implemented an IP fragmentation mechanism using `fragID`, `fragFlags`, and `fragOffset`. This allowed us to split the data into 100-byte chunks, ensuring reliable delivery over the acoustic medium.

### 4.3 Deep Dive into Fragmentation and Timing

A critical technical challenge arose when handling HTTP responses. Standard HTML source code for *example.com* spans approximately 822 bytes, which inherently exceeded

the maximum capacity of our 8-bit length field (`unsigned char`). This resulted in a modular overflow where the receiver misidentified the packet length as 54 bytes (822 (mod 256)), leading to persistent Cyclic Redundancy Check (CRC) failures.

To resolve this, we implemented an IP-layer fragmentation strategy. By partitioning the payload into 100-byte segments, we ensured compatibility with our frame structure. Furthermore, we discovered that acoustic transducers require a "physical cooldown" period. Transmitting packets in immediate succession caused inter-packet interference due to ambient echoes and speaker diaphragm inertia. We experimentally determined that a 400ms inter-segment delay was necessary to allow the acoustic environment to return to a baseline state, ensuring the preamble of the subsequent segment remained distinguishable.

## 4.4 ARP Implementation (Bonus Task 10)

We achieved dynamic address resolution. Before any data transmission, NODE1 broadcasts an ARP Request (`Type 50`). NODE2 responds with an ARP Reply (`Type 51`), allowing for automatic binding of IP addresses to the physical acoustic link.

## 4.5 ARP Protocol Interaction Analysis

The implementation of Task 10 marked our transition from static network configuration to dynamic node discovery. The process follows a strict request-reply cycle. Upon a `ping` command, the NODE1 client initiates an ARP lookup. If the target MAC (in this case, the specific acoustic link identifier) is absent from the `arpTable` cache, an `ARP_REQ` frame is transmitted.

As observed in our debug logs, NODE2 successfully parses the request and echoes back an `ARP_RSP` containing the identity string "I AM GATEWAY". This handshake not only validates the physical presence of the neighbor but also synchronizes the state machines of both nodes before the high-overhead ICMP or HTTP processes begin. This dynamic binding mechanism significantly reduced our reliance on pre-defined `config.txt` entries, making the Aethernet stack more adaptive to real-world hardware swapping.

# 5 Technical Challenges and Solutions

1. **UI Responsiveness:** Initially, our JUCE GUI froze when waiting for terminal input. We solved this by moving the `std::cin` logic to a background `std::thread`, ensuring the Message Thread remained free to refresh the window.
2. **Windows Socket Initialization:** We encountered issues where DNS resolution failed silently on Windows. We fixed this by explicitly calling `WSAStartup` in our gateway logic.
3. **Signal Overload:** We found that 100% system volume caused "clipping," which destroyed the CRC checksums. Through fine-tuning, we identified a "Golden Volume" of 80% that provided a perfect 100% similarity in file transfers.

## 6 Personal Reflections and Feedback

### 6.1 Solo vs. Teamwork

In my personal opinion, this project set is exceptionally cohesive and might actually be **better suited for a single student** rather than a group, provided the total workload is slightly reduced. Network protocol stacks require deep, end-to-end understanding; when split between members, the integration phase often becomes more difficult than the implementation itself.

### 6.2 Course Suggestions

The transition from Project 3 to Project 4 is quite steep. More documentation on how to bridge JUCE's audio callbacks with standard C++ networking libraries would be beneficial for future students.

## 7 Conclusion

The Aethernet project provided an invaluable hands-on experience in understanding the OSI model. By building a network from the samples up to the HTTP headers, we gained a profound respect for the complexity and elegance of the modern Internet.